

TRINETRA — Physical Junction Rig

Wiring Handoff Document — for an AI assistant guiding hands-on assembly

This document gives a hardware-focused AI assistant (used with no other project context) everything needed to walk a beginner through wiring the physical rig, step by step, safely. It is a HANDOFF document, not a request for new design decisions — every design choice below has already been made and confirmed with the project owner. If the assistant reading this wants to suggest a different approach than what's written here (different pins, different wiring topology, different lens layout), it should flag that as a question back to the user rather than silently substituting it — the firmware and backend code that will be written afterward depend on these exact choices.

1. Project context

TRINETRA is a college final-year major project: an AI-driven adaptive traffic signal control system. A microscopic traffic simulator (SUMO) drives a single simulated 4-way road junction; a machine-learning model predicts near-future traffic demand per lane; a rule-based decision engine picks which direction gets a green light; and a web dashboard shows all of this live. All of that already exists and works — it is pure software today.

The task now is to build a small PHYSICAL scale model of that same junction — an acrylic board with four traffic-signal mast heads — and make its real LEDs light up in sync with what the simulation's AI is deciding, live, over a USB cable. This is a hardware bring-up task: wiring an ESP32 microcontroller to a chain of addressable LEDs (WS2811 pixels), nothing more exotic than that. There is no WiFi, no cloud, no custom PCB — everything is breadboard/jumper-wire, and the only "smart" part is the software already written elsewhere.

The person you are helping is a beginner with electronics. Explain each step plainly, one instruction at a time, confirm it worked before moving to the next, and assume no prior knowledge of terms like breadboard rows, common ground, or pull-up resistors — define them briefly the first time they come up.

1.1 What is already done (do not redo this)

- Arduino IDE 2.x installed, with ESP32 (WROOM-32) board support and the Adafruit NeoPixel library.
- The ESP32's USB-to-serial chip (a Silicon Labs CP2102) has its Windows driver installed. It enumerates as a COM port (COM3 on the development machine so far; may differ on other USB ports/machines).
- The board has been proven to compile and flash code successfully (the stock "Blink" example uploaded and the onboard LED blinked).
- A bench-test Arduino sketch already exists for driving the pixels (full text in Appendix A) and is ready to flash as-is once wiring is done for one head.

None of the physical LED wiring has been done yet. That is the entire scope of this document.

2. Confirmed hardware design (do not change without asking)

The simulated junction has 4 approaches (North, South, East, West). Each approach has 3 lanes: a LEFT-turn-only lane, a STRAIGHT-only lane, and a RIGHT-turn-only lane. On this junction's traffic light program, the left-turn and straight movements ALWAYS run together in the same phase (never independently), while the right turn runs in its own separate phase. Because left and straight always show the same colour at the same time, they share ONE physical LED behind ONE combined arrow icon, instead of two separate LEDs — this saves wiring and matches how a real traffic signal would be built for this exact movement pattern.

2.1 Per-head lens layout — 4 LEDs per signal head

Position (left to right on the head)	What it shows	When it lights
Lens 1 — Red circle	A plain red circular light, shared by the whole approach	Lights red whenever NOTHING on this approach has a green light
Lens 2 — Amber circle	A plain amber circular light, shared by the whole approach	Lights amber only in the brief moment the approach's last green light ends
Lens 3 — Combined left+ahead arrow	ONE arrow icon with two arrowheads — one pointing left, one pointing ahead (see Appendix B for an example)	Lights green pointing left+ahead
Lens 4 — Right arrow	A single arrow pointing right	Lights to match the right-turn movement's own colour, independent of the other three

4 heads (one per approach) x 4 lenses each = 16 LEDs total for the whole rig.

2.2 Wiring topology — 4 independent chains, one per head

Each head's 4 LEDs are wired as their own short chain (a "strip" cut to exactly 4 pixels), and each of the 4 heads' chains connects to a SEPARATE pin on the ESP32 — NOT one long chain looping through all 4 heads. This was a deliberate choice: if one head's wire comes loose during a live demo, only that head goes dark; the other three keep working. Do not simplify this into a single 16-pixel daisy chain even though that would use fewer wires — it was explicitly rejected for reliability during a live demo.

Head (approach)	ESP32 GPIO pin (data)	Number of pixels in this chain
Head 1	GPIO 13	4
Head 2	GPIO 4	4
Head 3	GPIO 16	4
Head 4	GPIO 17	4

These 4 pins were chosen specifically because they are safe general-purpose, output-capable GPIOs on the ESP32 DevKit V1 board — they avoid the "strapping" pins (0, 2, 12, 15) which affect how the board boots, the flash pins (6–11) which are wired internally to the board's own flash chip, and input-only pins (34–39) which cannot drive an LED at all. If for any reason one of these 4 pins turns out to be unusable on the specific board in hand, ask before substituting a different pin — some pins listed above are reserved for other planned features (see 2.3).

2.3 Other pins already reserved (do not reuse these for LEDs)

Function	Pin	Status
Onboard link-activity LED	GPIO 2 (built into the board)	In use for the toolchain bring-up script
Countdown timer displays (optional future feature)	GPIO 14 (shared clock), GPIO 25 / 26 / 27 / 32 (one per approach)	Reserved, not yet wired
Emergency buzzer (optional future feature)	GPIO 33	Reserved, not yet wired

2.4 Data-line component per chain

Each of the 4 data lines needs a 470Ω resistor in series between the ESP32 pin and the pixel chain's DIN (data-in) wire — this protects the first pixel in each chain from a voltage spike when the board powers on. That means 4 resistors total, one per chain (this project's parts list already includes exactly 4 of these resistors for this reason).

The ESP32 outputs 3.3V logic; the WS2811 pixels are speced for roughly 5V logic. At this pixel count and wire length it very often works directly without any extra parts. IF, once wired, the colours glitch or show wrong/flickery colours: the fix is a single 1N4001 diode wired in series with the pixel chain's 5V supply line (not the data line) — this drops the supply voltage slightly so 3.3V logic reads clearly. Only add this if you actually see a problem; don't add it pre-emptively.

3. Power

Stage	What to power the LEDs from	Why
Bench-testing ONE head (4 pixels)	USB alone (ESP32's own 5V pin)	~4 pixels + the ESP32 itself draw roughly 150-200mA, which is within USB's limits
Full rig, all 4 heads wired (16 pixels)	A separate external 5V power supply feeding ONLY the pixels (the ESP32's own 5V pin)	ESP32's own 5V pin can only supply ~500mA, which is not enough for 16 pixels

Whichever power source is used, the pixel chain's GROUND wire must always be connected to the SAME ground as the ESP32's GND pin — even when using the external 5V supply, its ground and the ESP32's ground must be tied together. This is the single most common wiring mistake: forgetting the shared/common ground causes garbled or absent colours even though everything else looks correctly wired.

Brightness is capped at 60% in the firmware (already done in the code in Appendix A) specifically to reduce the worst-case current draw and reduce brownout risk — this does not need to be changed.

4. Step-by-step: wiring ONE head first

Do exactly one head at a time. Get one head fully working and tested before wiring the next — this makes it trivial to find a mistake (it can only be in the one thing just changed). Start with Head 1 (GPIO13).

- 1 Identify the 4-pixel cut segment for this head. It has 3 wires/pads: **5V** (power), **DIN** (data in), and **GND** (ground). There is a small arrow printed on the strip showing which way data flows — that arrow must point INTO this segment. If wiring several segments later, never connect two segments' DIN ends together, or DOUT to DOUT.
- 2 Push a 470Ω resistor into two separate rows of the breadboard.
- 3 Run a jumper wire from the ESP32's GPIO13 pin to the SAME breadboard row as one leg of the resistor.
- 4 Run a jumper wire from the resistor's OTHER leg to the segment's DIN wire.
- 5 Run a jumper wire from the ESP32's 5V (or VIN) pin to the segment's 5V wire.
- 6 Run a jumper wire from the ESP32's GND pin to the segment's GND wire.
- 7 Double-check: 3 wires total from the ESP32 to this segment (data-through-resistor, 5V, GND) — nothing else yet.

4.1 Flashing and testing

- 1 Open Arduino IDE, open the sketch in Appendix A (already written — do not rewrite its logic, only physically wire to match it).
- 2 Confirm near the top of the file: `#define HEADS_WIRED 1` — this must stay at 1 until more heads are wired.
- 3 Tools -> Board: the ESP32 Dev Module (or the specific ESP32 DevKit variant already selected previously). Tools -> Port: the COM port the board enumerates as (COM3 previously, but confirm in Device Manager if it differs).
- 4 Click Upload.
- 5 Expected result: the 4 LEDs on this head light up ONE AT A TIME, each cycling through red, then amber, then green, then moving to the next LED, repeating forever. Opening the Serial Monitor (115200 baud) also prints which LED and colour is active at each moment — this can be used to double check the physical order matches lens 1/2/3/4 as listed in section 2.1.

If nothing lights up: recheck the ground connection first (the most common fault), then confirm the DIN direction arrow, then confirm the correct GPIO pin was used.

Once Head 1 fully works, repeat the same wiring pattern for Head 2 (GPIO4), Head 3 (GPIO16), and Head 4 (GPIO17) — each is an independent repeat of the same 6 steps above, just on its own pin. Raise HEADS_WIRED to 2, then 3, then 4 as each new head is added, re-uploading each time.

5. Later stage: mounting behind the acrylic face (reference only)

This step comes AFTER all 4 heads are proven working loose on the breadboard. It is included here for context, not to be done yet.

Each head's acrylic face plate needs 4 apertures (holes) cut through a black acrylic front panel, matching the 4 lenses in section 2.1, left to right: a small circle (red), a small circle (amber), one COMBINED arrow shape with both a leftward and an upward arrowhead (see Appendix B for its exact outline), and a rightward arrow. A frosted/opal acrylic diffuser layer goes behind the face plate to spread each LED's point of light evenly across its aperture, and black internal baffles between each aperture's cavity stop one LED's light bleeding into its neighbour's aperture.

General head housing size used elsewhere in this project's design: roughly 32 x 95 mm face plate, about 25 mm deep housing, mounted on a post about 110 mm tall above the acrylic road board. These are a starting reference, not a hard requirement — the acrylic shop can adjust to fit the 4-aperture layout cleanly.

6. Firmware and software: not part of this handoff

Do NOT write or modify the logic that reads the live traffic simulation's signal state and drives these LEDs from it — that will be written separately, by the project owner's other AI assistant (Claude Code), once all 4 heads are wired and bench-tested and confirmed working with the test sketch in Appendix A. Your scope here is strictly: help wire the physical LEDs correctly, and help run the existing bench-test sketch as-is to confirm the wiring. If asked to go further than that, defer and say so.

Appendix A — current bench-test firmware (flash as-is)

This is the exact, already-written Arduino sketch for bench-testing the wiring. Copy it into the Arduino IDE exactly as shown (or use the existing file if working from the project folder, at `firmware/bench_test/bench_test.ino`).

```
// TRINETRA physical rig – bench test sketch (BOM v1.1 Section 8, adapted for
// the 4-lens-per-head design and 4 independent chains, Section 36).
//
// Purpose: prove each head's pixel chain works and is wired in the right
// order BEFORE any acrylic is cut. Does not talk to the backend yet – that
// is a separate sketch, once this one passes on every head you've wired.
//
// Design (Section 36, confirmed with the owner):
//   - 4 lenses per head: [0] red circle, [1] amber circle,
//   [2] combined left+ahead arrow, [3] right arrow.
//   - 4 heads, each its OWN chain on its OWN GPIO pin (fault isolation: a
//   loose wire on one head cannot take the others dark with it) –
//   GPIO13 / GPIO4 / GPIO16 / GPIO17, matching the BOM's 4 (not 1) 470ohm
//   resistors, one per data line.
//
// Wiring per head (bench only, per doc Section 5):
//   ESP32 <head's GPIO> -> 470ohm resistor -> that head's pixel chain DIN
//   Pixel chain 5V -> external 5V supply once testing more than a couple of
//   pixels at once (not the ESP32 5V pin) -- Section 5.3
//   Pixel chain GND -> shared with ESP32 GND -- must be common, or colors
//   glitch
//
// Only wire up as many heads as you've actually got pixels for right now –
// HEADS_WIRED below controls how many chains this sketch actually drives.
//
// If colors are wrong/flickery once wired: try the 1N4001 diode level-shift
// fallback in Section 5.4 before suspecting the code.

#include <Adafruit_NeoPixel.h>

#define LENSES_PER_HEAD 4
#define NUM_HEADS 4
// How many of the 4 heads are physically wired up right now. Start at 1
// while bench-testing a single cut segment; raise to 4 once all heads are
// wired. Never higher than NUM_HEADS.
#define HEADS_WIRED 1

const uint8_t HEAD_PIN[NUM_HEADS] = {13, 4, 16, 17};

Adafruit_NeoPixel heads[NUM_HEADS] = {
  Adafruit_NeoPixel(LENSES_PER_HEAD, HEAD_PIN[0], NEO_GRB + NEO_KHZ800),
  Adafruit_NeoPixel(LENSES_PER_HEAD, HEAD_PIN[1], NEO_GRB + NEO_KHZ800),
  Adafruit_NeoPixel(LENSES_PER_HEAD, HEAD_PIN[2], NEO_GRB + NEO_KHZ800),
  Adafruit_NeoPixel(LENSES_PER_HEAD, HEAD_PIN[3], NEO_GRB + NEO_KHZ800),
};

// Cap brightness per Section 9's brownout mitigation (~60%).
#define BRIGHTNESS 153 // 60% of 255

void showAllOff() {
  for (int h = 0; h < HEADS_WIRED; h++) {
    heads[h].clear();
    heads[h].show();
  }
}

void setup() {
  Serial.begin(115200);
  for (int h = 0; h < HEADS_WIRED; h++) {
    heads[h].begin();
    heads[h].setBrightness(BRIGHTNESS);
  }
  showAllOff();
  Serial.print("TRINETRA bench test starting, heads wired: ");
  Serial.println(HEADS_WIRED);
}

// Cycles every lens on every wired head through red, amber, green, off – one
```

```

// lens at a time. Watch for aperture bleed once heads are assembled behind
// acrylic (Section 3's baffle note); on the open bench there is nothing to
// bleed into yet, this just proves addressing and color are both correct.
void loop() {
  for (int h = 0; h < HEADS_WIRED; h++) {
    for (int i = 0; i < LENSES_PER_HEAD; i++) {
      showAllOff();
      Serial.print("Head ");
      Serial.print(h);
      Serial.print(" lens ");
      Serial.print(i);
      Serial.println(": RED");
      heads[h].setPixelColor(i, heads[h].Color(255, 0, 0));
      heads[h].show();
      delay(600);

      Serial.print("Head ");
      Serial.print(h);
      Serial.print(" lens ");
      Serial.print(i);
      Serial.println(": AMBER");
      heads[h].setPixelColor(i, heads[h].Color(255, 120, 0));
      heads[h].show();
      delay(600);

      Serial.print("Head ");
      Serial.print(h);
      Serial.print(" lens ");
      Serial.print(i);
      Serial.println(": GREEN");
      heads[h].setPixelColor(i, heads[h].Color(0, 255, 0));
      heads[h].show();
      delay(600);
    }
  }

  // All-on white pulse only if you're deliberately checking the worst-case
  // current draw from Section 5.6 with a multimeter in line. Comment out
  // otherwise -- it's the highest-risk moment for a brownout on USB power.
  // for (int h = 0; h < HEADS_WIRED; h++) {
  //   heads[h].fill(heads[h].Color(255, 255, 255));
  //   heads[h].show();
  // }
  // delay(1000);
  // showAllOff();
}

```

Appendix B — the combined left+ahead arrow's exact shape

This is the precise outline used for the combined-arrow lens in the project's own 3D software model, so the physical acrylic aperture can match it exactly: one shared stem forking into an upward arrowhead and a leftward arrowhead. It looks like a capital "h" shape with two arrowheads — a vertical arrow pointing up, with a horizontal arrow branching off to the left partway up the stem.

Outline points (a closed polygon, unitless — scale proportionally to whatever final aperture size the acrylic face plate uses; for reference, keeping the numbers below as millimetres directly produces an icon a little over 20 mm wide and 19 mm tall, a reasonable match to a face plate aperture of that scale):

Point #	x	y	description
1	-0.08	-0.45	stem, bottom-left
2	0.08	-0.45	stem, bottom-right
3	0.08	0.16	stem's right edge going straight up
4	0.22	0.16	out to the up-arrowhead's base
5	0.00	0.50	UP TIP (top point of the icon)
6	-0.22	0.16	up-arrowhead's other base corner
7	-0.08	0.16	back in to stem width
8	-0.08	0.03	down to where the left branch forks off
9	-0.34	0.03	out along the left branch's top edge
10	-0.34	0.15	flare up for the left-arrowhead's base
11	-0.55	-0.05	LEFT TIP (leftmost point of the icon)
12	-0.34	-0.25	left-arrowhead's other base corner
13	-0.34	-0.13	back in to the left branch's width
14	-0.08	-0.13	back to the stem, closes the shape

In plain terms for a designer without vector-editing familiarity: draw an upward-pointing arrow (a narrow rectangle with a wide triangular arrowhead on top). Partway up the rectangle's left edge, add a second, smaller arrow branching off to the left (a narrower horizontal rectangle ending in a triangular arrowhead pointing left). The two shapes share the same central stem so the whole thing reads as one connected icon, not two separate arrows placed side by side.

Appendix C — full parts list for this stage

Item	Qty needed for bench testing	Notes
ESP32 DevKit V1 (WROOM-32)	1	Already confirmed working
WS2811 addressable pixel string, 5V, pre-wired connectors	16 pixels used, cut into 4 segments of 4	Cut only at the strip's marked cut points
Full-size breadboard, 830 point	1	
Jumper wires (M-M, M-F, F-F)	As needed	~3 per head for the bench test stage
470 ohm resistor	4	One per head's data line
1N4001 diode	0-2	Only if colours glitch once wired (Section 2.4)
5V 2A wall adapter or USB power bank	1	Only needed once testing more than ~1 head at a time
USB data cable (confirmed data-capable, not charge-only)	1	Already in use, confirmed working